

CA1 – Open Source Firewalls

Network Security

BSc. Computing in Digital Forensics and Cyber Security (TU863)

Submission Date: 28th February 2026

Lecturer: Peter Alexander

Students: Chinedu Nkem(B00162316)
Favour Godson(B00163920)

1. Introduction

Firewalls are a core component of network security, used to monitor and control traffic based on a defined set of rules. Without one, a machine is openly exposed to reconnaissance, service exploitation, and denial of service attacks. This report covers two open source firewalls, nftables and OPNsense, looking at how each one is configured and tested in a virtual lab environment. The firewalls are compared based on their effectiveness, ease of use, and how well they mitigate the attacks carried out during testing.

2. Nftables – Background

Nftables is a Linux kernel subsystem used for packet filtering and classification. It has been available since kernel version 3.13, released in January 2014, and is now the default firewall on major distributions including Debian 10 and Red Hat Enterprise Linux 8 (Wikipedia, 2025). It was developed by the Netfilter project to replace the older iptables, ip6tables, arptables, and ebtables tools, combining all of their functionality into a single tool (Red Hat, 2024).

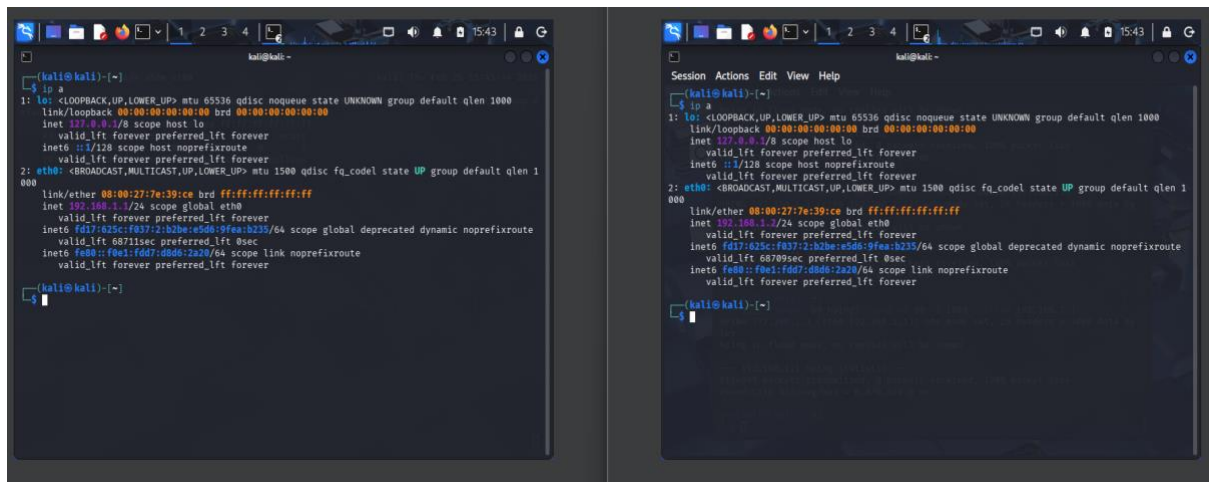
The structure of nftables is built around tables, chains, and rules. A table is a container that holds chains, and chains hold the individual rules that decide what happens to packets. One key difference from iptables is that nftables does not create any default tables or chains, meaning the administrator has full control over everything that is configured (Arch Wiki, 2025). The inet address family allows a single ruleset to handle both IPv4 and IPv6 traffic at the same time, which is a significant improvement over the older approach of needing separate tools for each protocol.

Nftables is configured using the nft command line tool. Rules can be added directly from the terminal or loaded from a configuration file stored at /etc/nftables.conf. This file-based approach makes it easy to save, restore, and manage rulesets consistently (Ubuntu Security Documentation, 2026).

3. Testbed Setup

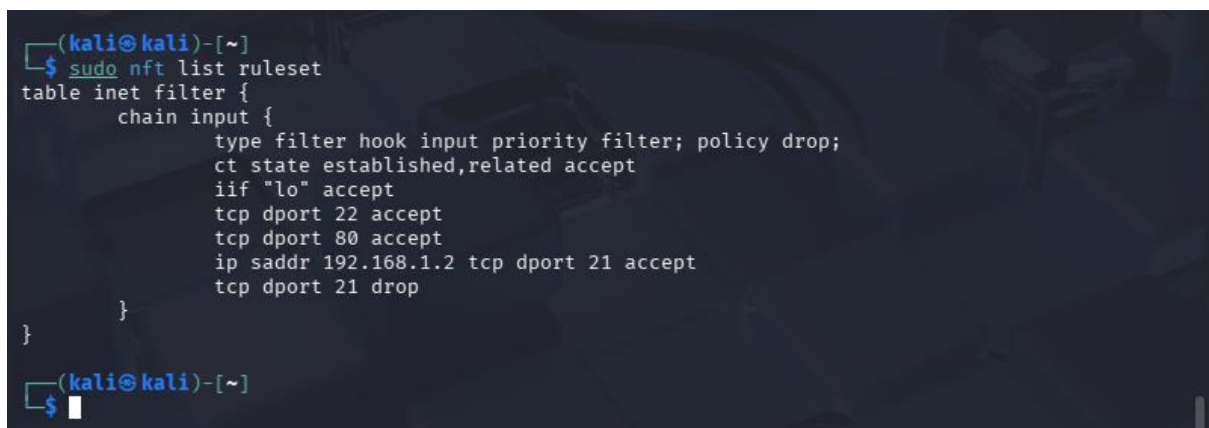
The nftables testbed was built using two virtual machines in Oracle VirtualBox, both running Kali Linux. Kali was chosen because nftables, Nmap, and other security tools come pre-installed. The first VM was used as the firewall and target machine, and the second acted as the attacker. Both VMs were placed on the same internal network with no external internet access, creating an isolated environment for testing.

The firewall VM was assigned the IP address 192.168.1.1 and the attacker VM was given 192.168.1.2. For the service access control test, the attacker VM's IP was temporarily changed to 192.168.1.3 to simulate a machine coming from outside the trusted subnet. Three services were started on the firewall VM before any tests were run: Apache web server on port 80, SSH on port 22, and an FTP server on port 21. This gave the scanner meaningful results to discover and made the before and after comparison more visible.



4. Nftables Configuration

The nftables ruleset was set up using the inet table family so that both IPv4 and IPv6 traffic could be handled by the same rules. The input chain was created with a default drop policy, meaning any traffic that does not match an explicit allow rule is automatically blocked. This is known as a default deny approach and is considered best practice in firewall configuration (Ubuntu Security Documentation, 2026). Base rules were added first to allow loopback traffic and established connections before any service specific rules were applied.



5. Testing

5.1 Test 1 – Port Scanning and Service Detection

Port scanning is typically one of the first steps an attacker takes when targeting a system. Using Nmap, an attacker can find out which ports are open, what services are running, and the exact software versions. This information can then be used to search for known vulnerabilities. According to the official Nmap documentation, every open port on a system is a potential avenue for attack (Lyon, 2024).

Before any firewall rules were applied, an Nmap SYN scan and service version scan were run from the attacker VM. The commands used were `nmap -sS 192.168.1.1` and `nmap -sV 192.168.1.1`. The results clearly showed all three services running on the target with their software versions exposed. This is exactly the kind of information an attacker would use to plan a targeted exploit.

```

(kali@kali)-[~]
$ nmap -sV 192.168.1.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-26 16:05 EST
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --sys-
tem-dns or specify valid servers with --dns-servers
Nmap scan report for 192.168.1.1
Host is up (0.00040s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp?
22/tcp    open  ssh      OpenSSH 10.2p1 Debian 2 (protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.65 ((Debian))
MAC Address: 08:00:27:7E:39:CE (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 11.72 seconds

```

After applying the firewall rules, the same scan was run again from the trusted IP 192.168.1.2. Ports 22 and 80 remained open as expected since they were explicitly allowed in the ruleset. Port 21 was still visible from this address because the access control rule permits the trusted IP to connect to it.

However, as you can see from the screenshot below. The ip address was changed from a trusted ip (192.168.1.2) to an untrusted ip address(192.168.1.3) and the port 21 was denied visual access as long as the other 998 filtered tcp ports.

```

(kali@kali)-[~]
$ sudo ip addr del 192.168.1.2/24 dev eth0
$ sudo ip addr add 192.168.1.3/24 dev eth0
[sudo] password for kali:

(kali@kali)-[~]
$ nmap -sV 192.168.1.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-26 16:21 EST
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --sys-
tem-dns or specify valid servers with --dns-servers
Nmap scan report for 192.168.1.1
Host is up (0.00068s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.2p1 Debian 2 (protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.65 ((Debian))
MAC Address: 08:00:27:7E:39:CE (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

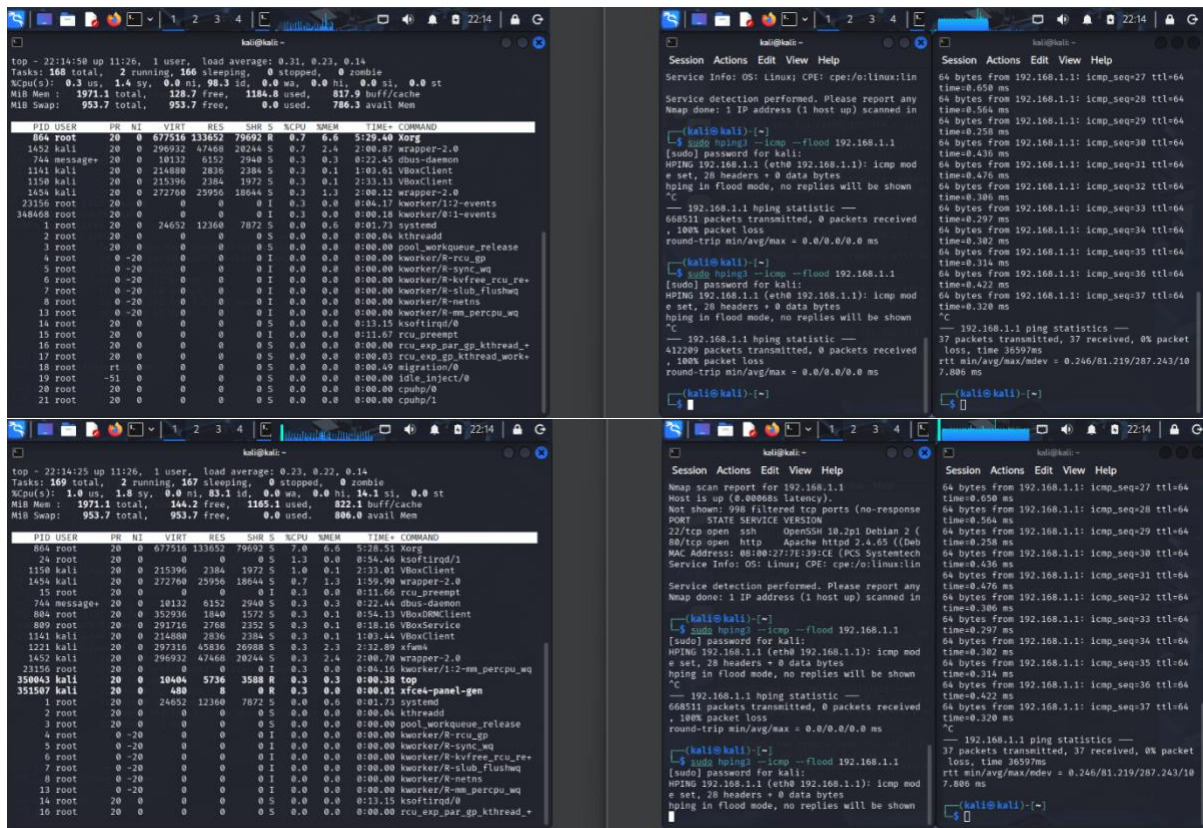
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 12.02 seconds

```

5.2 Test 2 – ICMP Flood Under Extended Attack

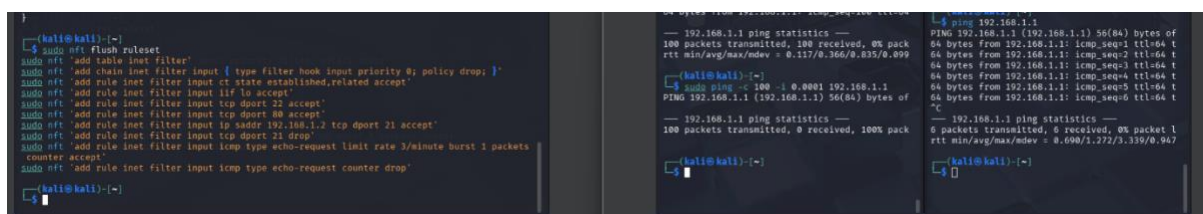
This test idea was gotten from NSS Labs generation firewall test methodology, which talks about how a properly configured firewall should be able to block malicious traffic(ICMP flood) while also allowing actual legitimate traffic to pass through so this led me to carry out a ICMP flood attack where as I using hping to from my attacker vm to flood my Target machine while I also had another terminal open which was just sending in normal pings.

Before I applied and firewall rules I used the command 'hping3 --icmp --flood 192.168.1.1'.which approximately sent 668,511 packet in about 36 seconds. I also ran a second ping from the second terminal. The normal ping was successful with 37 packets sent and 0 percent packet loss. Just with these results it was clear to say that system could not distinguish between what was good traffic and what was bad traffic.



I even went further ahead to gather proof of the CPU strain that the system collected from the ICMP flood. Before the flood the CPU idle was at 98.6% and software interrupts was at 0. During the flood the CPU idle was dropped to 83.1% and software interrupts jumped to 14.1%. This shows the system was forced to accept this useless traffic, process it and think of a reply for it to send back which slowed down the CPU. Obviously, this is a lab test so it won't hurt anyone's pc but if I was to send more traffic, I could entirely disrupt all traffic from that device

After I applied ICMP rate limiting rule, I ran the flood attack again alongside with the normal ping. configured the rule to only allow 3 ICMP echo requests per minute with a burst of 1 packet. The flood sent 100 packets and achieved 100% packet loss meaning that every single packet was dropped by the firewall before reaching the target. Meanwhile a normal ping was able to successfully send 6 packets and they were all received with no packet loss. This demonstrates that the firewall successfully blocked the flood attack entirely while continuing to pass legitimate traffic without interruption,



6. OPNsense – Background

OPNsense is an open source firewall and routing platform based on FreeBSD. It was created in 2015 as a fork of pfSense and is developed by Deciso B.V. Unlike nftables which is purely command line based, OPNsense provides a web based graphical interface which makes it easier to configure and manage without needing to know specific syntax or commands.

OPNsense includes a wide range of features such as stateful packet inspection, NAT, traffic shaping, VPN support and an intrusion detection system. It is commonly used in small to medium sized businesses and home lab environments as a full network security solution. Because it runs as a dedicated firewall appliance rather than just a set of rules on a Linux machine, it is better suited to scenarios where the firewall needs to sit between two separate networks and control all traffic passing through.

7. OPNsense – Configuration

For this assignment, OPNsense was deployed across three virtual machines in Oracle VirtualBox. The first VM ran the OPNsense firewall itself, acting as the middleman/gateway between the other two machines. The second VM was used as the target victim machine, placed on the LAN side of the network. The third VM was the attacker machine, placed on the WAN side.

To simulate a real life scenario, OPNsense was configured with two network adapters. The first adapter (em0) was attached to an internal network named LAN and assigned the IP address 192.168.1.1/24. The second adapter (em1) was attached to a separate internal network named WAN and assigned the IP address 192.168.2.1/24. The target victim machine was given the address 192.168.1.3/24 on the LAN side, and the attacker machine was given the address 192.168.2.2/24 on the WAN side.

This setup meant that any traffic from the attacker had to pass through OPNsense to reach the target, which is exactly how a perimeter firewall would function in a real environment. Since the VMs were on isolated internal networks with no DHCP server, IP addresses and routes had to be configured manually on each Kali machine at the start of every session. Services including SSH, Apache and a simulated FTP service using netcat were started on the target VM before each test to ensure there was something for the attacker to scan.

```
--- 192.168.2.2 ping statistics ---
3 packets transmitted, 3 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 0.854/1.123/1.467/0.256 ms

Press ENTER to continue.

*** OPNsense.internal: OPNsense 26.1.2_5 (amd64) ***

LAN (em0)      -> v4: 192.168.1.1/24
WAN (em1)     -> v4: 192.168.2.1/24

HTTPS: SHA256 09 DF B6 81 6C 30 0F 98 AE 65 CC 62 21 A1 A6 CA
              4E DC 6B F0 95 3F 77 DE 05 AB CB 46 DF 1C D5 FB

0) Logout                      7) Ping host
1) Assign interfaces           8) Shell
2) Set interface IP address   9) pfTop
3) Reset the root password    10) Firewall log
4) Reset to factory defaults  11) Reload all services
5) Power off system           12) Update from console
6) Reboot system              13) Restore a backup

Enter an option: █
```

8. Testing

8.1 Test 1 – Port Scanning and Service Detection

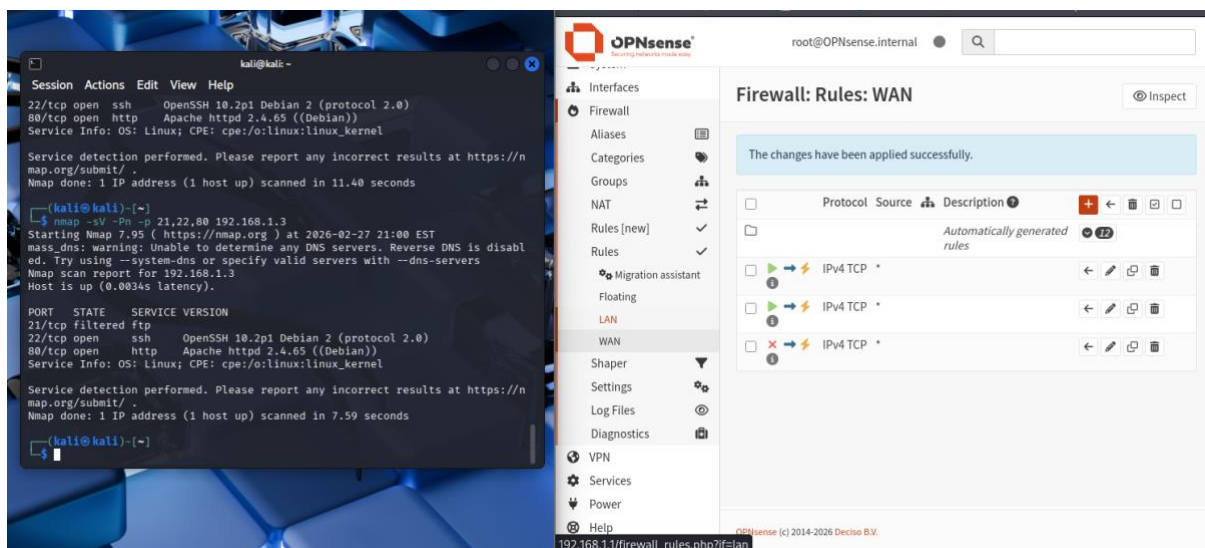
Before any firewall rules were configured on OPNsense, it was important to first establish what an attacker could see from the outside. Port scanning is commonly used in the early stages of an attack to gather information about a target system.

An Nmap scan was run from the attacker VM using the command `nmap -sV -Pn -p 21,22,80 192.168.1.3`. The reason I specifically chose to just scan those ports because I deliberately opened them on the target VM and because the network now had to pass through a firewall, scanning all 1000 ports took way too long. The scan returned results showing port 21 open and simulated using netcat, port 22 running OpenSSH 10.2p1 and port 80 running Apache 2.4.65. With no rules in place OPNsense was allowing all traffic through meaning the target was fully exposed to the attacker.

```
kali@kali: ~  
Session Actions Edit View Help  
Host is up.  
All 1000 scanned ports on 192.168.1.3 are in ignored states.  
Not shown: 1000 filtered tcp ports (no-response)  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 209.49 seconds  
  
(kali@kali)-[~]  
$ nmap -sV -Pn -p 21,22,80 192.168.1.3  
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-27 20:05 EST  
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid servers with --dns-servers  
Nmap scan report for 192.168.1.3  
Host is up (0.0029s latency).  
  
PORT      STATE SERVICE VERSION  
21/tcp    open  ftp?        
22/tcp    open  ssh       OpenSSH 10.2p1 Debian 2 (protocol 2.0)  
80/tcp    open  http      Apache httpd 2.4.65 ((Debian))  
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 11.40 seconds  
  
(kali@kali)-[~]  
$
```

Three rules were added to OPNsense through the web GUI. Two pass rules were created to allow TCP traffic on port 22 and port 80, and a block rule was added for port 21. Once the rules were applied the same Nmap scan was run again using `nmap -sV -Pn -p 21,22,80 192.168.1.3`.

The results were noticeably different. Port 21 now showed as filtered meaning OPNsense was successfully dropping that traffic before it could reach the target. Ports 22 and 80 remained open and still returned their service version information. This shows that OPNsense was able to selectively block unwanted services while keeping legitimate ones accessible, which is exactly the behaviour expected from a properly configured firewall.

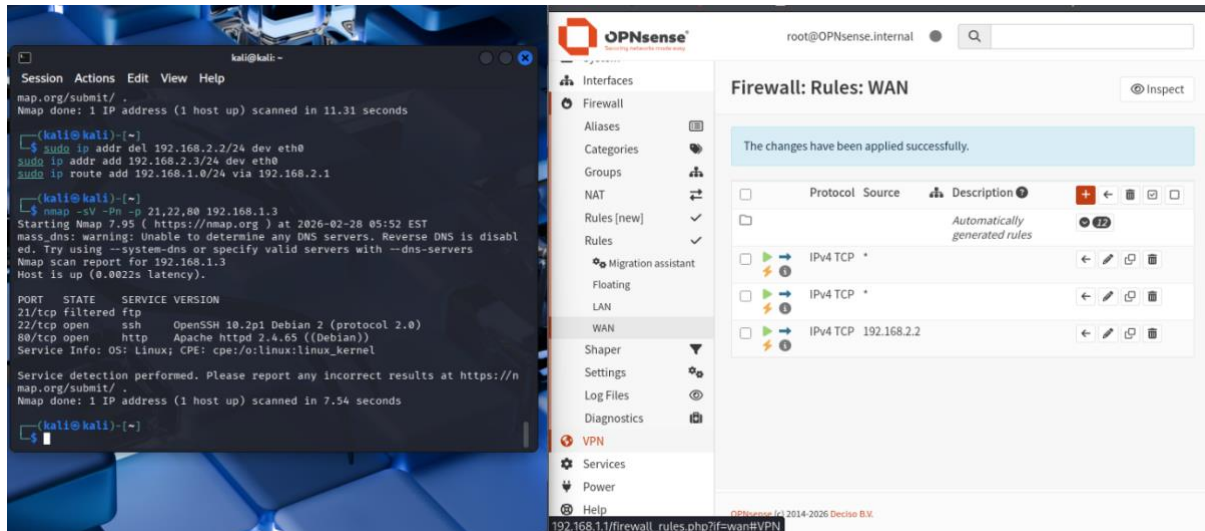


8.1 Test 2 Service access Control by IP Address

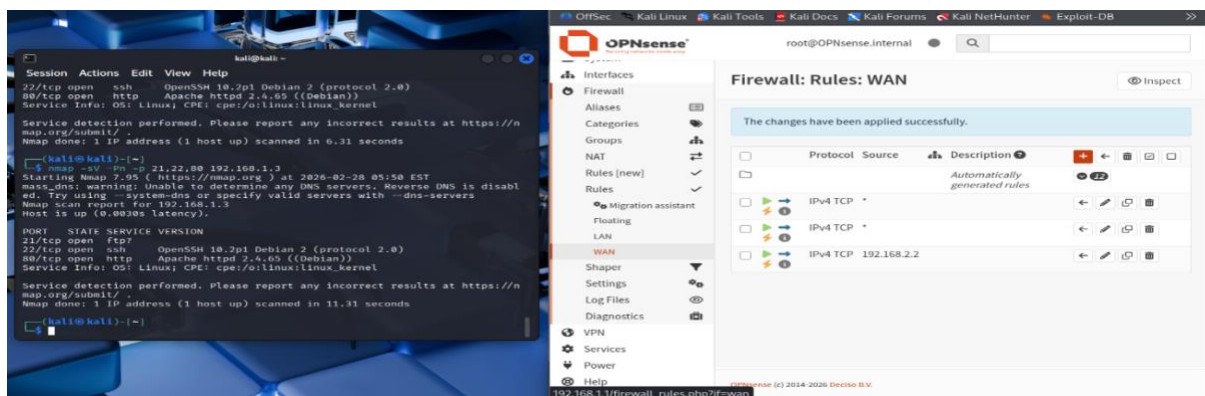
Not all firewall rules need to block a service entirely. In some cases it makes more sense to restrict access to a service based on where the request is coming from. This is useful in situations where a service like FTP should only be accessible to a specific trusted machine and blocked for everyone else.

To test this, a pass rule was added in OPNsense to allow TCP traffic on port 21 only from the trusted IP address 192.168.2.2. No other source was permitted to access that port.

When the Nmap scan was run from 192.168.2.2 using `nmap -sV -Pn -p 21,22,80 192.168.1.3`, port 21 showed as open confirming that the trusted machine could reach the FTP service.



The attacker IP was then changed to 192.168.2.3 to simulate a different untrusted machine on the same network. Running the same scan from 192.168.2.3 showed port 21 as filtered, meaning OPNsense was blocking that traffic before it reached the target.



This demonstrates that OPNsense can enforce granular access control based on source IP, allowing administrators to expose services only to specific trusted hosts while keeping them hidden from everyone else.

9. Comparison of Nftables and OPNsense

9.1 Configuration and Ease of Use

One of the most noticeable differences between the two firewalls was how rules were configured. Nftables required manual rule creation through the command line using tables, chains and rule syntax. While this provided a high level of control, it required precise knowledge of networking concepts and correct syntax. Misconfiguration could easily result in unintended traffic blocking.

In contrast, OPNsense provided a webbased interface which allowed firewall rules to be created and modified visually. During the pass and block rules for ports 21, 22 and 80 were implemented quickly through the GUI without needing command syntax. This made configuration more accessible and reduced the likelihood of human error.

9.2 Network Architecture and Deployment

The nftables firewall was deployed directly on the host machine, which situated the firewall and the protected services on the same system. Although this configuration is effective for host protection it diverges from the typical architecture employed in actual enterprise deployments.

In contrast, OPNsense operated as a dedicated perimeter firewall, strategically positioned between the Wide Area Network and the Local Area Network. Consequently, all traffic originating from potential attackers was compelled to traverse the firewall prior to reaching the targeted machine. This configuration more accurately copies real-world firewall deployments and makes a clearer difference between trusted and untrusted networks.

9.3 Effectiveness Against Port Scanning

Both firewalls successfully reduced information disclosure during Nmap scans. With nftables, ports were selectively visible depending on whether the source IP was trusted or untrusted, demonstrating very strong access control capabilities.

Similarly, OPNsense successfully filtered port 21 traffic while allowing ports 22 and 80 to remain accessible. In both cases, unwanted services appeared as filtered rather than open, indicating that packets were dropped before reaching the target.

This shows that both firewalls were equally effective at mitigating reconnaissance attacks when properly configured.

9.4 Flexibility and Administrative Control

Nftables offers greater flexibility and fine-grained control because rules are written directly at kernel packet filtering level. Advanced configurations such as rate limiting and custom logic can be implemented precisely.

OPNsense prioritises usability and centralized management. Features such as logging, rule ordering, and interface separation which are easier to manage visually, making it suitable for administrators who require strong security without deep command-line expertise.

9.5 Overall Comparison

In summary, nftables provides powerful low-level control and excellent performance for host-based firewall protection, particularly when advanced customization is required. OPNsense, however, offers a more realistic network firewall solution with easier management and clearer separation of network zones.

Both firewalls proved effective in mitigating scanning and unauthorized access attempts but they serve different operational roles: nftables is better as a configurable host firewall whereas OPNsense functions best as a dedicated perimeter security appliance.

10. Conclusion

This project aimed to evaluate and compare the effectiveness of two open source firewall technologies, nftables and OPNsense, through implementation within a realistic virtualised network environment and the execution of practical security tests. Both firewalls were configured to protect systems from common network threats including ports scanning, unauthorised access attempts and denial of service style traffic.

The testing demonstrated that both solutions were capable of successfully enforcing access control policies and mitigating malicious traffic when correctly configured. Nmap scanning results showed that each firewall was able to reduce exposed attack surfaces by filtering or blocking unauthorised ports. Additionally, nftables rate-limiting rules effectively mitigated ICMP flood traffic, significantly reducing system resource consumption compared to an unprotected system. OPNsense similarly prevented unwanted traffic from reaching internal systems through its stateful inspection and interface-based rule enforcement.

Through this implementation and testing process, the study demonstrates the critical role firewalls play in modern network security and reinforces the importance of layered defence strategies combining both host-based and perimeter protection mechanisms.

References

Arch Wiki (2025) Nftables. Available at: <https://wiki.archlinux.org/title/Nftables>

Debian Wiki (2025) Nftables. Available at: <https://wiki.debian.org/nftables>

Ubuntu Security Documentation (2026) Nftables. Available at: <https://documentation.ubuntu.com/security/security-features/network/firewall/nftables/>

Caury, I. (2025) The Complete nftables Guide: Modern Linux Firewall Mastery. Medium. Available at: <https://medium.com/@ihouelecaury/the-complete-nftables-guide-modern-linux-firewall-mastery-79fb86894d5c>

Red Hat (2024) Getting started with nftables. Red Hat Enterprise Linux 8 Documentation. Available at: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/8/html/configuring_and_managing_networking/getting-started-with-nftables_configuring-and-managing-networking

OPNsense Documentation (2026) Firewall. Available at: <https://docs.opnsense.org/firewall.html>

Deciso B.V. (2026) *OPNsense: Open Source Firewall*. Available at:
<https://opnsense.org>

Home Network Guy (2024) *How to Configure Firewall Rules in OPNsense*.
Available at: <https://homenetworkguy.com/how-to/configure-opnsense-firewall-rules/>

OPNsense Documentation (2026) *Initial Installation & Configuration*. Available at:
<https://docs.opnsense.org/manual/install.html>

Home Network Guy (2022) *How to Install and Configure OPNsense*. Available at:
<https://homenetworkguy.com/how-to/install-and-configure-opnsense/>